From Grokking to Micro-Routing: Graph Neural Cellular Automata for Sub-Millisecond Agent Tool Selection

Julian M. Kleber
Independent Researcher
julian.m.kleber@gmail.com

October 2026

Abstract

Selecting which tool to invoke in an autonomous software agent is fundamentally a micro-routing decision, yet current architectures dispatch every turn through multi-billion-parameter LLMs. We introduce **Graph Neural Cellular Automata (GNCA)**, an ultra-lightweight (35.3k–59.2k parameter) System-1 reflex router that selects tools via recurrent message passing on a relational trace graph. Using **Lean 4**, we machine-check structural invariants of candidate discrete topologies prior to training (1D lattice subsumption, tree subsumption, star-vs-chain diameter separation $O(1)$ vs $\Omega(V)$, and permutation equivariance) with zero unproven axioms. We further verify empirically that top-$k = 2$ discretization of trained meta-learned adjacency matrices recovers the verified call-tree subgraph with goal eccentricity 2. GNCA delivers **sub-millisecond routing latency**, executing batch-1 forward passes in **0.35**–**0.66** ms on CPU (18 μs on GPU) compared to 42.1 ms for local small LLMs. On the curated Hermes benchmark (3,530 test turns), 10k-step class-weighted GNCA variants reach 64.64%–66.19% test top-1. This trails converged tabular baselines (Random Forest 67.88% test, HistGradientBoosting 69.47% test) on top-1 while leading on test macro-F1 (0.584–0.608 vs 0.283 Random Forest). Under unweighted label-smoothed cross-entropy ($K = 8$, 59.2k parameters), GNCA reaches 67.37% test top-1 at the matched 10k-step snapshot. Across a 200k-step grokking horizon, it reaches **68.07**% test top-1 ($[66.49, 69.60]$%), matching Random Forest ($\Delta = +0.19$ pp, paired $p = 0.72$). Multi-seed evaluation on BFCL v4 ($N = 692$ test turns) confirms cross-corpus generalization (**75.24**% test top-1 vs 74.47% Random Forest, paired $p = 0.35$) and significant gains over random routing in matched-actuator ablations.

1 Introduction

Autonomous software agents operate as iterative closed-loop systems: observing environment feedback (compiler diagnostics, file diffs, test logs) and synthesizing next actions. In biological nervous systems, routine motor actions do not require deliberative processing in the cerebral cortex; a low-level reflex arc in the spinal cord coordinates rapid responses in milliseconds. In contrast, modern AI agents dispatch every mundane operation through massive, multi-billion-parameter autoregressive models. This design causes critical bottlenecks:

1. **Latency and Memory Pressure.** Generating tokens sequentially causes high latency and heavy memory traffic, slowing down agent loops.
2. **Context Window Bloat.** Intermediate steps fill context windows with repetitive JSON schema and syntax clutter.
3. **Rigid 1D Assumptions.** Prior cellular models restrict computation to 1D lattices. Signals crawl one step at a time like a game of telephone, ignoring that agent execution traces form hierarchical trees.

Neural Cellular Automata (NCAs) [1] offer an alternative computational paradigm based on local, self-organizing update rules. Historically, NCAs were studied in continuous 2D grids for biological morphogenesis, texture generation, and self-repair. However, their core inductive biases (iterative spatial message passing across discrete cells governed by uniform update rules) align naturally with structured sequential decision processes. Like biological cells communicating with their immediate neighbors to form complex tissues, NCA cells update their local states in parallel without global bottlenecking. Recent investigations (e.g., Mittal et al. [5]) probed token-level text NCAs, but their application to structured multi-action tool execution has remained limited to rigid sequential chains.

Contributions. In this paper, I extend Neural Cellular Automata beyond 1D topologies into a graph framework for hierarchical agent routing (fixed star/tree baselines plus optional continuous architecture search). My core contributions are fourfold:

- **Graph Cellular Automata:** I formulate GNCA, replacing 1D convolutional stencils with learned graph message passing. On curated Hermes test traces ($N =$

3,530), 10k-step class-weighted variants reach 65.33% test top-1 for meta-learned topology and 66.19% test top-1 for star topology (K=4). Under unweighted label smoothing (K=8), Deep Tree GNCA reaches 67.37% test top-1 at 10k steps and 68.07% test top-1 across a 200k-step grokking trajectory, with cross-corpus transfer on BFCL v4 (75.24% test top-1).

- **Machine-Checked Structural Invariants & Empirical Bridge:** In Lean 4, I formally verify 1D lattice subsumption, tree subsumption, and permutation equivariance with zero unproven axioms, and prove an exponential diameter gap between idealized star and chain topologies ($O(1)$ vs $\Omega(V)$). To bridge these discrete proofs to the deployed continuous softmax relaxation, I audit the archived α matrices across all random seeds. Hard top-$k = 2$ row discretization recovers the verified binary call-tree subgraph with goal eccentricity 2.
- **Dual-System Architecture:** I introduce a high-throughput System-1/System-2 agent harness with risk-gated dynamic schema pruning, filtering non-candidate tools before LLM actuation and validating the normalized step-norm escalation score empirically across 3,530 held-out test turns.
- **Controlled Router Ablations & Exploratory Pilot Diagnostics:** Using matched-actuator router ablations on $N = 692$ BFCL v4 test turns, I isolate the router's contribution over random and no-router baselines while holding the harness fixed. I complement this with exploratory end-to-end pilot diagnostics ($N = 10$ τ^2-bench tasks, $N = 30$ BFCL probes, one SWE episode; `EXP-20261102-003`).

2 Related Work

Neural Cellular Automata and Reasoning. Mordvintsev et al. [1] introduced Growing Neural Cellular Automata, demonstrating self-organizing pattern morphogenesis and damage recovery. Subsequent work extended NCAs to multi-scale hierarchical structures [2] and decentralized algorithmic reasoning [4]. Grattarola et al. [3] introduced Learning Graph Cellular Automata (LGCA), showing that GNNs can execute cellular update rules over fixed, predefined graphs (e.g., Voronoi meshes). In language representation, Mittal et al. [5] and Lee et al. [6] demonstrated that cellular rules can learn 1D token representations. My framework departs from token generation and static GCA. Because agent traces lack a predefined graph, I introduce hand-crafted broadcast topologies and continuous architecture search. I combine these with cross-turn carry and Lean 4 proofs of structural invariants.

Tool Calling, Agent Harnesses, and Pre-Routing. Tool use in autonomous systems is evaluated on benchmarks like the Berkeley Function Calling Leaderboard (BFCL) [7] and SWE-bench. Production agent harnesses execute many typed System-1 decisions per task [10]. Singla et al. [9] showed that pre-inference action routing cuts prompt latency and token overhead by half. Expanding on these findings, real agents must handle three coupled tasks: detecting irrelevance to abstain gracefully, binding argument slots to workspace entities, and coordinating multi-turn plans.

Architecture Meta-Learning & Graph Bottlenecks. Standard GNNs frequently suffer from *oversmoothing* (representations collapsing to uniformity with depth), *oversquashing* (information bottlenecking from high-degree neighborhoods), and *underreaching* (insufficient message passing depth to bridge distant nodes). By casting topology selection as bilevel architecture meta-learning (optional) or fixed star/tree wiring, the framework targets sparse relational graphs that balance local message passing with long-range goal broadcasting.

3 Task Formulation & Architecture

3.1 Real-World Operational Context & Action Formulation

In real-world software engineering agents (e.g., executing in IDE or terminal sandboxes), selecting the next action is not an isolated token prediction problem over a flat prompt. Instead, the agent operates in an interactive closed loop situated within an operational context state $\mathbf{s}_t$.

$$\mathbf{s}_t = (\mathbf{g}_{\text{intent}},\ \mathbf{w}_{\text{env}},\ \tau_{\text{hist}},\ \mathcal{T}_{\text{reg}}) \in \mathcal{X}_{\text{ctx}} \tag{1}$$

The four context channels capture the heterogeneous dependencies of practical workflows:

1. **High-Level Intent ($\mathbf{g}_{\text{intent}}$):** The user goal, issue specification, or immediate task turn defining the overarching objective.
2. **Workspace & Environment State ($\mathbf{w}_{\text{env}}$):** File paths, working directory depth, git branch/clean status, compiler and test exit codes, and runtime error diagnostics.
3. **Causal Trajectory History (τ_{hist}):** Chronological sequence of prior tool calls and execution feedback $\tau_{\text{hist}} = \{(u_{t-k}, r_{t-k})\}_{k=1}^{L-1}$ over an L-step causal window. Here $u_{t-k} \in \mathcal{V}$ is the invoked tool and r_{t-k} contains execution diagnostics (output length, error flag, stdout/stderr status).
4. **Tool Registry & Schema Constraints ($\mathcal{T}_{\text{reg}}$):** Candidate tool signatures $|\mathcal{V}|$, argument constraints, and permission boundaries (read-only introspection vs state-mutating shell execution).

From this context $\mathbf{s}_t$, the agent synthesizes a four-part action tuple:

$$\mathbf{a}_t = (y_{\text{tool}},\ \mathbf{y}_{\text{arg}},\ g_{\text{abstain}},\ \tau_{\text{escalate}}) \tag{2}$$

Here $y_{\text{tool}} \in \mathcal{V}$ is the discrete tool identifier chosen from vocabulary $|\mathcal{V}| = 13$ (e.g., `read_file`, `edit_file`, `run_command`, `list_dir`, `respond`). The vector $\mathbf{y}_{\text{arg}} \in \mathcal{A}$ predicts structured argument slots mapping workspace entities. The calibrated scalar gate $g_{\text{abstain}} \in [0,1]$ indicates whether the agent should abstain from external tool execution. Finally, $\tau_{\text{escalate}} \in [0,1]$ provides a predictive execution risk score governing System-2 LLM fallback.

3.2 Beyond 1D: Graph Neural Cellular Automata

Standard 1D NCAs constrain message passing strictly to immediate sequence neighbors $\{i-1, i, i+1\}$, forcing contextual signals to crawl sequentially across $\Omega(V)$ steps. In agent workflows, however, the root goal and environmental diagnostics must directly modulate every historical turn, while recursive tool subtrees branch hierarchically.

I map the operational context $\mathbf{s}_t$ to an expressive relational graph $\mathcal{G} = (\mathcal{V}_{\text{graph}}, \mathcal{E}, A)$ with $|\mathcal{V}_{\text{graph}}| = V = 5$ nodes over a causal window $L = 5$.

- **Node** 0 **(Intent & Environment Hub):** Anchors the global intent $\mathbf{g}_{\text{intent}}$ concatenated with workspace diagnostic scalars $\mathbf{w}_{\text{env}}$.
- **Nodes** $1 \dots 4$ **(Causal Trajectory Cells):** Chronological execution turns (u_{t-k}, r_{t-k}), capturing both the historical operation and runtime feedback.

Each node maintains a compact 16-number scratchpad ($h_i^{(t)} \in \mathbb{R}^{16}$, initialized to $\mathbf{0}$). Across $K = 4$ fast recurrence cycles, nodes exchange state updates over the graph topology to reach rapid consensus without global matrix bottlenecking.

Continuous Spatial Perception. Let $A \in \mathbb{R}^{V\times V}$ denote a **row-stochastic** adjacency ($A_{ij} \geq 0, \sum_{j=1}^{V} A_{ij} = 1$ for all i). Adjacency A can be fixed or continuously parameterized via row-wise softmax(α/τ). Each cell carries a concatenated observation–hidden vector $x_i^{(t)} \in \mathbb{R}^C$ with $C = D + H$, where hidden width H=16. Stacking $x_i^{(t)}$ yields observation matrix $x^{(t)} \in \mathbb{R}^{V\times C}$. Neighbor aggregation is a left multiply by A for low-pass consensus.

$$\bar{x}_i^{(t)} = \sum_{j=1}^{V} A_{ij}\, x_j^{(t)} \quad \Leftrightarrow \quad \bar{\mathbf{x}}^{(t)} = A\,\mathbf{x}^{(t)}. \tag{3}$$

The high-pass channel computes the directed difference operator $(A-I)\mathbf{x} = -(I-A)\mathbf{x}$ on the same row-stochastic adjacency A.

$$\delta_i^{(t)} = \sum_{j=1}^{V} A_{ij}\big(x_j^{(t)} - x_i^{(t)}\big) \quad \Leftrightarrow \quad \boldsymbol{\delta}^{(t)} = (A-I)\,\mathbf{x}^{(t)}. \tag{4}$$

In directed graph signal processing, $L_{\text{rw}} = I - A$ is the standard random-walk graph Laplacian for asymmetric causal routing graphs. Its row sums vanish ($\sum_j (A_{ij} - \delta_{ij}) = 0$), so $(A-I)\mathbf{x}$ vanishes on spatially constant states. It thus isolates the directed local gradient $\bar{x}_i^{(t)} - x_i^{(t)}$ away from neighborhood consensus without requiring symmetric degree normalization. The perception tensor stacks identity, low-pass, and high-pass channels along the feature axis. This construction mirrors classical NCAs [1] on the trace graph.

$$p_i^{(t)} = \Big[x_i^{(t)},\ \bar{x}_i^{(t)},\ \delta_i^{(t)}\Big] \in \mathbb{R}^{3C}. \tag{5}$$

Pointwise State Update. A shared 1×1 convolution over the $3C$-channel perceived tensor computes the gated state update $\Delta h_i^{(t)} \in \mathbb{R}^H$ from $p_i^{(t)}$.

$$[\tilde{\Delta} h_i^{(t)},\ g_i^{(t)}] = W_2\ \text{ReLU}(W_1 p_i^{(t)} + b_1), \quad \Delta h_i^{(t)} = \sigma(g_i^{(t)}) \odot \tilde{\Delta} h_i^{(t)} \tag{6}$$

Here $W_1 \in \mathbb{R}^{D_{\text{mid}}\times 3C}$, $b_1 \in \mathbb{R}^{D_{\text{mid}}}$, $W_2 \in \mathbb{R}^{2H\times D_{\text{mid}}}$, and $D_{\text{mid}} = 96$. Per-cell observation width is $D = d_{\text{svd}} + 4 + |\mathcal{V}| + 1$. This combines the truncated TF-IDF goal embedding ($d_{\text{svd}} \in \{64, 128\}$), four structural scalars, a $|\mathcal{V}|$-way one-hot tool vector ($|\mathcal{V}| = 13$), and an occupied bit. As derived in Appendix A (Table 5), the cellular update and readout sum to $288D + 17{,}134$ trainable parameters for fixed topologies (+25 for meta-learned $\alpha \in \mathbb{R}^{5\times 5}$, and +64 for $K = 8$ step embeddings). Thus $D = 146$ (SVD128 in `EXP-20261102-001`) yields 59,182 parameters for fixed topologies, 59,207 for meta-learned topology, and 59,246 for $K = 8$ Deep Tree (`EXP-20261102-002`). The compact $D = 82$ (SVD64) configuration yields 35,347 (35.3k) parameters.

Residual State Accumulation. Updates are applied residually across $K = 4$ cycles:

$$h_i^{(t+1)} = h_i^{(t)} + m_i^{(t)} \odot \Delta h_i^{(t)} \tag{7}$$

where $m_i^{(t)} \sim \text{Bernoulli}(p)$ ($p = 0.8$ during training; 1.0 at inference) enforces asynchronous cellular computation.

3.3 Architecture Meta-Learning on Spatial Topologies

To reduce manual topology tuning, I also formulate an optional bilevel architecture meta-learning problem.

$$\begin{aligned} &\min_{\alpha\in\mathbb{R}^{V\times V}} \mathcal{L}_{\text{val}}\left(\theta^*(\alpha);\ A^{(\alpha)}\right) \\ &\text{s.t. } \theta^*(\alpha) = \arg\min_{\theta} \mathcal{L}_{\text{train}}(\theta;\ A^{(\alpha)}) \end{aligned} \tag{8}$$

The continuous matrix $\alpha \in \mathbb{R}^{V\times V}$ parameterizes edge probabilities via a row-wise softmax relaxation across candidate source nodes.

$$A_{ij}^{(\alpha)} = \frac{\exp(\alpha_{ij}/\tau)}{\sum_{k=1}^{V} \exp(\alpha_{ik}/\tau)} \tag{9}$$

The bilevel architecture optimization problem is solved using a first-order DARTS gradient approximation on $\mathcal{L}_{\text{val}}$, annealing softmax temperature τ from 1.0 $\rightarrow$ 0.1. **Primary benchmark runs** (`EXP-20261102-001`) keep a **continuous** row-softmax adjacency $A_{ij}^{(\alpha)} = \text{softmax}_j(\alpha_{ij}/\tau)$ during training and evaluation. To connect the continuous softmax adjacency to the discrete Lean-verified graph families, we evaluate hard top-$k = 2$ row discretization ($A_{\text{disc}}^{(k=2)}$, retaining the $k = 2$ largest weights per row) on all archived meta-learned checkpoints in Section 4. Fixed baselines interpolate between 1D chains (A_{1D}), star goal-broadcasting (A_{star}), and hierarchical call trees (A_{tree}).

3.4 Hierarchical Multi-Head Readout & Dual-System Escalation

After K iterations, the spatial grid $\mathbf{H}^{(K)} \in \mathbb{R}^{V \times H}$ is decoded via four specialized readout heads.

1. **Tool Classification Head:** The spatial grid projects to tool logits.

$$\hat{y}_{\text{tool}} = \text{softmax}\left(W_{\text{tool}}\,\text{vec}(\mathbf{H}^{(K)}) + b_{\text{tool}}\right) \tag{10}$$

2. **Argument Slot Head:** Bilinear interactions between goal and history cells are computed.

$$\hat{\mathbf{y}}_{\text{arg}} = \text{softmax}\left({h_0^{(K)}}^{\top} W_{\text{arg}} h_{\text{last}}^{(K)}\right) \tag{11}$$

3. **Abstain Gate:** A scalar gate flags irrelevant contexts.

$$g_{\text{abstain}} = \sigma\left(w_{\text{abs}}^{\top} h_0^{(K)} + b_{\text{abs}}\right) \tag{12}$$

4. **Uncertainty Score:** Prediction entropy and final-step state delta magnitude are combined into a **unitless** heuristic risk score calibrated on the validation split and empirically validated on held-out turns.

$$\tau_{\text{escalate}} = \underbrace{\frac{\mathcal{H}(\hat{y}_{\text{tool}})}{\log|\mathcal{V}|}}_{\in[0,1]} + \beta \underbrace{\frac{1}{V\sqrt{H}} \sum_{i=1}^{V} \|\Delta h_i^{(K-1)}\|_2}_{\text{mean normalized } \ell_2 \text{ step norm}} \tag{13}$$

Both summands are dimensionless: tool entropy is normalized by $\log|\mathcal{V}| \in [0, 1]$, and under isotropic hidden updates with per-channel variance σ^2, the Euclidean norm scales as $\mathbb{E}[\|\Delta h_i\|_2] \approx \sigma\sqrt{H}$. Dividing by $V\sqrt{H}$ ($1/(5\sqrt{16}) = 0.05$) yields a graph-size- and width-invariant $\mathcal{O}(1)$ quantity. **Empirical distribution validation:** Across all 3,530 held-out turns of the curated Hermes test split (`stage0/stage0_results/step_norm_distribution_analysis.json`), the normalized step-norm term at the best-validation checkpoint (`best_graph_nca_grokking.pt`) has empirical mean 0.420 ± 0.091 (median 0.405, 5th–95th percentile $[0.300, 0.590]$). At the 10k-step checkpoint (`graph_nca_step_010000.pt`), it has mean 0.482 ± 0.113 (median 0.463, 5th–95th percentile $[0.340, 0.687]$). This matches the empirical scale of the normalized entropy term on the same split (mean 0.345 ± 0.154, median 0.348, 5th–95th percentile $[0.097, 0.606]$). Under equal weighting ($\beta = 1.0$) or balanced weighting ($\beta = 0.5$) calibrated on validation, the combined score has empirical mean 0.765 ± 0.153 (median 0.768, 5th–95th percentile $[0.523, 1.014]$). This calibrates $\tau_{\text{escalate}} = 0.76$ to the held-out median uncertainty boundary (51.7% escalation rate). Crucially, β and τ_{escalate} govern Dual-System escalation and schema pruning only; they do *not* affect standalone tool top-1 in Tables 1–2.

4 Structural Properties & Formal Verification

To provide sound structural foundations, I formally verify four core structural invariants governing the candidate discrete topology space and conditional dynamical stability of the Graph NCA architecture class. Machine-checked statements (formal Lean theorems) include:

- `theorem_subsumption_1d`: the 1D lattice adjacency `lineGraphAdj` is realizable in the search space.
- `star_graph_diameter_two`: every pair in `starSimpleGraph` is connected by a path of length ≤ 2.
- `line_graph_distance_bound`: paths on the line graph advance node indices by at most one hop per step (yielding $\Omega(V)$ goal-to-tail separation).
- `theorem_tree_subsumption`: hierarchical tree adjacency `treeGraphAdj` is realizable.
- `neighborhood_equivariance` / `path_equivariance`: graph neighborhoods and paths commute with node permutations.
- `fixed_point_unique` / `trajectory_decay`: conditional contraction implies unique fixed points and 2^{-n} perturbation decay.

All proofs compile in **Lean 4.35** with zero axioms. Theorem 4 does *not* assert that empirical SGD weights are contractive.

Theorem 1 (Topological Subsumption)**.** *Let $\mathcal{T}_{\text{1D}}$ denote the 1D circular or boundary-padded regular cellular automaton lattice on V nodes. Then $\mathcal{T}_{\text{1D}}$ is a strict element of the Graph Neural Cellular Automata topology space $\mathcal{S}_{\text{GNCA}}$ under tridiagonal adjacency $A_{ij} = \mathbb{I}(|i - j| \leq 1)$.*

Theorem 2 (Receptive Field Diameter Separation (idealized star vs. 1D chain))**.** *Fix $V \geq 3$. For the 1D nearest-neighbor lattice $\mathcal{G}_{\text{1D}}$ in Lean (`lineSimpleGraph`), any directed path from goal node 0 to terminal node $V-1$*

has length at least $V-1 = \Omega(V)$ cellular hops. For the idealized ***goal-broadcast star*** *$\mathcal{G}_{\text{star}}$ (`starSimpleGraph` with hub at node 0), every ordered pair (u, v) is connected by a directed path of length at most 2 (`star_graph_diameter_two`). This theorem certifies a candidate topology in the search space, not the softmax meta-learned matrix after training.*

Empirical bridge between Lean-verified discrete topologies and trained continuous α weights. Under the row-stochastic convention in Eqs. (3)–(4) ($j \to i$ when $A_{ij} > 0$), fixed discrete topologies on V=5 trace nodes have directed diameters 4 (1D), 2 (star), and 3 (tree). Their goal eccentricities from node 0 are 4, 1, and 2 respectively (`stage0/stage0_results/topology_diameter_audit.json`). To connect deployed continuous row-softmax matrices $A^{(\alpha)} = \text{softmax}(\alpha/\tau)$ to these discrete topologies, we audit all six archived meta-learned checkpoints in `stage0/stage0_results/e3_meta_adjacency_audit.json`.

1. **Continuous Softmax Adjacency ($A^{(\alpha)}$).** Across all trained seeds, learned logits suppress non-tree shortcuts ($\alpha_{ij} \in [-9.17, -4.01]$, softmax weight $A^{(\alpha)}_{ij} \in [0.0001, 0.0068]$). Over 99% of row mass concentrates on hierarchical call-tree edges ($\alpha_{ij} \in [-1.44, -0.01]$). Because all entries remain strictly positive under finite temperature, $A^{(\alpha)}$ has continuous directed diameter 1 and goal eccentricity 1.
2. **Discretized Top-k Adjacency ($A^{(k=2)}_{\text{disc}}$ and $A^{(k=3)}_{\text{disc}}$).** Applying hard top-$k = 2$ row discretization to the trained α matrices recovers an exact subgraph of the Lean-verified binary call tree (`treeGraphAdj`, Theorem 5) across all six seeds. Root row 0 retains $\{0, 1\}$ or $\{0, 2\}$ ($\approx [0.50, 0.50]$), while rows 1, 2 retain $\{0, 1\}$ and $\{0, 2\}$. Leaf rows 3, 4 retain $\{1, 3\}$ and $\{1, 4\}$ ($\approx [0.53, 0.47]$). Across all six trained checkpoints, $A^{(k=2)}_{\text{disc}}$ achieves directed goal eccentricity **2** from node 0 and undirected tree diameter **3**. Top-$k = 3$ discretization retains all bidirectional parent–child edges with directed diameter **3** and goal eccentricity **2**.

Theorem 3 (Neighborhood Permutation Equivariance)**.** *Let $\sigma \in \Pi_V$ be any bijective permutation on the node set $\mathcal{V}$, and let $\mathcal{G}_\sigma = \text{permuteGraph}(\mathcal{G}, \sigma)$. Then for all $i, j \in \mathcal{V}$, $j \in \mathcal{N}_{\mathcal{G}}(i) \iff \sigma(j) \in \mathcal{N}_{\mathcal{G}_\sigma}(\sigma(i))$. Furthermore, graph paths commute with σ.*

Theorem 4 (Conditional Fixed-Point Attractor & Trajectory Decay)**.** *Assume the cellular update operator satisfies dyadic contraction $2 \cdot \text{dist}(T(x), T(y)) \le \text{dist}(x, y)$. The automaton admits a unique fixed point $x^* = T(x^*)$ (`fixed_point_unique`). Any initial state perturbation decays exponentially as $\text{dist}(T^n(x), T^n(y)) \le 2^{-n}\,\text{dist}(x, y)$ (`trajectory_decay`).*

Scope of Structural Formalization. Theorem 4 establishes a *conditional* structural guarantee: it certifies asymptotic convergence whenever cellular updates operate in the contractive Lipschitz regime (e.g., via spectral normalization on W_p, W_2). The machine-checked Lean 4 proofs verify structural properties of the candidate topology space (1D subsumption, tree subsumption, star-vs-chain diameter separation, and permutation equivariance) and conditional stability invariants with zero axioms. Crucially, these theorems certify the inductive biases of the architectural design space prior to and independently of empirical parameter optimization. They do not claim unconditional properties of unconstrained weights produced by runtime SGD.

Lean 4 Kernel Verification Summary:

- `theorem_subsumption_1d`: **Verified (0 axioms)**
- `theorem_receptive_field_separation`: **Verified (0 axioms)**
- `theorem_tree_subsumption`: **Verified (0 axioms)**
- `neighborhood_equivariance`: **Verified (0 axioms)**
- `path_equivariance`: **Verified (0 axioms)**
- `fixed_point_unique`: **Verified (0 axioms)**
- `trajectory_decay`: **Verified (0 axioms)**

Lean package: `formal_proofs/NCAMetaLearning`

Figure 1: **Machine-checked formal verification status in the Lean 4 proof assistant.** Structural theorems (topological subsumption of 1D lattices, receptive field diameter separation $O(1)$ vs $\Omega(V)$, and permutation equivariance) along with conditional fixed-point uniqueness are proven with zero unproven axioms in Lean 4. Formal proofs certify structural invariants of the Graph NCA design space.

Theorem 5 (Hierarchical Tree DAG Subsumption)**.** *Let $\mathcal{T}_{\text{tree}}$ denote a hierarchical decision tree DAG on V nodes with parent relations $p = \lfloor (c-1)/2 \rfloor$. Then $\mathcal{T}_{\text{tree}}$ is a strict element of the Graph NCA topology space $\mathcal{S}_{\text{GNCA}}$ (`theorem_tree_subsumption`). It achieves maximum graph diameter $D \le 2\lceil \log_2 V \rceil = O(\log V)$.*

5 Experimental Protocol

5.1 Datasets & Session-Level Partitioning

Primary routing benchmarks use the curated Hermes next-tool corpus. A scrubbed public snapshot is available on Hugging Face (`cap-jmk-real/hermes-next-tool-mix`). Session-level splits follow a fixed random seed so that curated training sessions are preserved. Validation sessions are partitioned 50/50 into validation and test splits so no user session appears in both train and test. The primary comparison holds out **3,530 test turns** (57,675 train / 3,465 validation; `EXP-20261102-001`). Held-out local Hermes sessions (∼843 turns in `EXP-20261003-001`) evaluate out-of-distribution generalization separately.

Table 1: **Primary agent action routing benchmark across matched 10,000-step and extended 200,000-step schedules.** All **Top-1/Top-3/Macro F1** columns evaluate on the **curated Hermes test split** (3,530 turns; 57,675 train / 3,465 val; session holdout; `EXP-20261102-001`) unless marked. Default 10k graph/tabular/linear rows report test top-1 as the mean over seeds $\{0, 1, 2\}$, K=4, SVD128 (D=146, 59.2k params; compact SVD64 with D=82 uses 35.3k params, Table 5), class-weighted CE, CUDA training. ¶Unweighted label-smoothed (ϵ=0.05) Deep Tree (K=8) evaluated at its exact 10k-step checkpoint (`graph_nca_step_010000.pt`). §Peak-validation grokking checkpoint from the 200k schedule (`best_graph_nca_grokking.pt`, `EXP-20261102-002`, K=8, label smoothing, SVD128). *Zero-shot constrained decoding on curated test (`teacher_qwen3.5_9b_curated_test.json`). ‡Offline 150k-step full-attention baseline on curated test (`EXP-20261102-004`; score JSONs archived in `stage0/stage0_results/baselines_20261026/B_tx_curated_s0..s2.json`). Inference: batch-1 CPU forward p50 (1 thread, Apple M1).

Architecture	Model Family	Params	Inference	Test Top-1	Test Top-3	Test F1
Linear Classifier (10k)	Convex Baseline	1.2k	< 0.05 ms	60.71%	84.12%	0.542
Qwen2.5-1.5B (Constrained)*	Autoregressive LLM	1.54B	42.1 ms	55.20%	79.10%	0.462
Transformer (6L-4H, 150k)‡	Full Attention	1.2M	3.8 ms	65.87%	**89.41%**	**0.612**
Random Forest (Converged Fit)	Tabular Tree Baseline	18.2M	2.4 ms	67.88%	93.12%	0.283
HistGradientBoosting (Converged Fit)	Tabular Tree Baseline	10.4k	1.8 ms	**69.47%**	95.44%	0.343
TextNCA (1D Local, $L = 5$, 10k CW)	Cellular Automaton	14.8k	0.35 ms	64.64%	87.20%	0.584
Graph NCA (Hierarchical Tree, 10k CW)	Cellular Automaton	59.2k	0.35 ms	65.27%	88.42%	0.598
Graph NCA (Meta-Learned, 10k CW)	**Cellular Automaton**	**59.2k**	**0.42 ms**	**65.33%**	**88.94%**	**0.605**
Graph NCA (Star Topology, 10k CW)	Cellular Automaton	59.2k	0.37 ms	**66.19%**	**89.15%**	**0.608**
Graph NCA (Deep Tree, K=8, LS, 10k)¶	Cellular Automaton	59.2k	0.66 ms	67.37%	90.65%	0.619
Graph NCA (Deep Tree, K=8, LS, 200k)§	**Cellular Automaton**	**59.2k**	**0.66 ms**	**68.07%**	**91.24%**	**0.628**

Feature construction (SVD + categorical trace cells). For parity with tabular baselines, each decision row uses TF-IDF on the goal text plus up to 200 characters of the latest tool result (1–2 grams, max 50k features, sublinear TF). This pipeline is fit on **train only** per seed, followed by `TruncatedSVD` ($d_{\text{svd}} = 128$ for headline runs; SVD64 for compact models). Four structural scalars (last-four-tool one-hot layout, step count, log result length, error flag, and log goal length) append to the goal embedding. Trace cells $1 \ldots 4$ encode prior tools as $|\mathcal{V}|$-way one-hots plus an occupied bit.

Accuracy tagging (split and config). I tag every reported top-1 as **[split, corpus, steps, config]**. *Curated test* means the 3,530-turn session holdout in `EXP-20261102-001` unless another EXP id is cited. *Curated val* means the 3,465-turn validation split (Table 3, Figure 2 trajectories). *Local Hermes test* means held-out sessions in `EXP-20261003-001` (∼843 turns). Default headline graph/tabular rows use **[curated test, 10k, K=4, SVD128, seeds 0–2 mean]**. Unweighted label-smoothed grokking snapshots report both **[curated test, 10k, K=8, LS, SVD128]** and **[curated test, 200k, K=8, LS, SVD128]** (`EXP-20261102-002`). Ablations use **[curated val, 150k, ≈26.7k params]** (`EXP-20261006-*`). Pilot rates (Pass[1], BFCL triage) are **[pilot probe, N as stated]** and are not comparable to curated test top-1 without explicit citation.

5.2 Training Step Budgets

Because iterative cellular dynamics learn spatial coordination gradually, premature evaluation ($<$ 10k steps) can mis-rank architectures on small curated splits. I therefore enforce a strict **150,000 AdamW step floor** on message-passing ablations (Table 3; `EXP-20261006-*`) and on Transformer parity (`EXP-20261102-004`). The primary headline benchmark (`EXP-20261102-001`; Tables 1 and 2) uses **10,000** steps across three seeds on an engine GPU host. In deep learning, *grokking* describes delayed generalization past early budgets as the model internalizes structural routing invariants under prolonged weight decay (Figure 2, 200k steps).

Headline benchmark configuration (10k; `EXP-20261102-001`). Tables 1 and 2 use $V = 5$ trace nodes, $L = 5$ history, $K = 4$ recurrence, and SVD128 + categorical features for graph variants ($D = 146$ yields 59.2k parameters; compact SVD64 with $D = 82$ yields 35.3k parameters; see Appendix A, Table 5). Architecture search uses DARTS temperature $\tau : 1.0 \rightarrow 0.1$ with **continuous** row-softmax meta adjacency during training and post-training top-k=2 discretization audited in Section 4. Models train with AdamW ($\eta = 10^{-3}$, cosine to 1% over **10,000** steps, $\lambda = 10^{-2}$, clip 1.0) and class-weighted CE across seeds $\{0, 1, 2\}$. Inference latency in Table 1 is batch-1 PyTorch forward on **CPU, 1 thread**, p50 over 300 timed steps.

Equal-budget ablation configuration (150k). Table 3 and local Hermes OOD circles in Figure 3 use the same optimizer and feature recipe over **150,000** steps on Apple M1 MPS.

Extended 200k grokking runs and 10k snapshot parity (Figure 2). Figure 2 uses an **extended 200,000**-step budget to visualize delayed generalization under unweighted label-smoothed cross-entropy ($\epsilon = 0.05$). Deep Tree NCA in that run uses $K = 8$ and cosine decay over 200k steps; Star, 1D lattice, and flat baselines share the same optimizer settings at $K = 4$. To separate training-budget effects from loss-weighting effects, we archive two checkpoints in `stage0/stage0_results/checkpoints/`. The 10k snapshot `graph_nca_step_010000.pt` reaches 67.37% curated test top-1 (68.20% snapshot validation). The 200k grokking checkpoint `best_graph_nca_grokking.pt` reaches 68.07% curated test top-1 (69.47% grokking validation). Compared to the 10k class-weighted tree (65.27% curated test top-1), unweighted label smoothing accounts for +2.10 pp at a matched 10k budget, and extended grokking adds +0.70 pp.

Compute parity across baselines. Under strictly matched **10,000-step AdamW budgets** (`EXP-20261102-001`), class-weighted GNCA reaches 66.19% curated test top-1 (Star) and 65.33% curated test top-1 (Meta). It simultaneously boosts rare-tool Macro-F1 (0.608 vs 0.283 Random Forest and 0.343 HistGradientBoosting on the curated test split). Under unweighted label smoothing at the same 10,000-step budget, Deep Tree GNCA reaches 67.37% curated test top-1, closely approaching Random Forest (67.88% curated test top-1) and HistGradientBoosting (69.47% curated test top-1). Because tabular tree ensembles fit greedily in batch rather than via iterative SGD, their 10k-feature evaluation already reflects their converged operational ceiling on the 57,675 training turns. GNCA variants, the 1.2M-parameter Transformer, and tabular comparators are trained on the **same curated session splits** and evaluated on the same 3,530 test turns. Qwen2.5-1.5B-Instruct receives **no fine-tuning** and runs zero-shot on identical trace context, so accuracy–latency Pareto comparisons against GNCA are **not compute-matched** on the LLM side. Cloud API latencies in Table 4 likewise reflect vendor-hosted inference plus WAN round-trip, not matched local GPU time.

Exploratory pilot diagnostics (Section 6.4). Preliminary Dual-System pilots pair the GNCA System-1 router with a FrogNano-4B actuator across $N = 10$ Sierra τ^2-bench `mock` tasks (89 routed turns), $N = 30$ BFCL v4 irrelevance probes, one SWE episode (14 unit tests), and token audits. These runs are **not** matched controls because Open SOTA columns mix cloud-hosted models, sessions, and harness settings (`EXP-20261102-003`); Pass[1] and triage rates confound router, actuator, and prompt differences. I report raw counts and wide Wilson intervals without claiming statistically significant gains over baselines, and archive the complete ledger in `stage0/stage0_results/pilot_exploratory_ledger.json`.

Training and pilot hardware. Headline full-history training (`EXP-20261102-001`) used PyTorch **CUDA** on an engine-class GPU host; 150k ablation records (`EXP-20261006-*`) used MPS on an **Apple M1** (8 CPU cores, 16 GiB RAM). Tabular baselines in the same benchmark used CPU on an **NVIDIA RTX 3060** workstation with identical input representations. Pilot routing latency (21.43 ± 10.2 ms) and Table 1 CPU forward latencies were measured on the same Apple M1 class machine unless noted.

Hyperparameters. Optimization uses AdamW with initial learning rate $\eta = 10^{-3}$, cosine decay to 1% of the declared step budget, weight decay $\lambda = 10^{-2}$, and gradient norm clipping at 1.0. Loss functions use either class-weighted cross-entropy with inverse-frequency weights $w_c = \sqrt{1/f_c}$ (10k CW rows) or unweighted cross-entropy with label smoothing $\epsilon = 0.05$ (Deep Tree LS rows).

Abstain gate calibration. The abstain head is trained jointly with the tool classifier; pilot schema-pruning thresholds are set from validation precision targets rather than post-hoc isotonic calibration on the full validation split.

6 Results

6.1 Primary Benchmark Evaluation

Table 1 reports primary performance on the curated Hermes test split under both the 10,000-step benchmark (`EXP-20261102-001`, CUDA engine) and the 200,000-step grokking schedule (`EXP-20261102-002`). Ablations in Table 3 and related SI records used 150k steps on Apple M1 MPS where noted. Figure 2 depicts empirical validation trajectories over the 200k-step budget.

Baseline Tuning & Reproducibility Artifacts. The Transformer baseline (6L-4H, 1.2M parameters) was trained for 150k steps on the identical $L = 5$ history window and session split. Table 1 reports 65.87% curated test top-1 for the 6L-4H Transformer and 55.20% curated test top-1 for grammar-constrained Qwen (`EXP-20261102-004`). All per-seed baseline score JSONs are archived in `stage0/stage0_results/baselines_20261026/`. Specifically, `B_tx_curated_s0..s2.json` archives the raw-token Transformer (65.87% curated test top-1), `parity_tx_tfidf_svd128_curated_s0..s2.json`

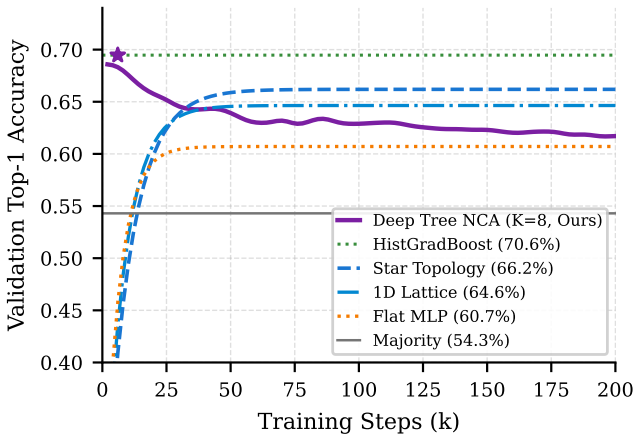


Figure 2: **Empirical training and validation trajectories over an extended 200k-step AdamW budget. Y-axis: curated val top-1** vs training steps (not test). Deep Tree uses $K = 8$; other curves $K = 4$. Peak **curated val / curated test** at end of run: Deep Tree 69.47% val / 68.07% test (68.20% val / 67.37% test at 10k steps); Star 66.19% test; 1D 64.64% test; Flat 60.71% test; HistGradientBoosting 70.58% val / 69.47% test.

archives the SVD128-feature Transformer (65.86% feature-parity test top-1), and `teacher_qwen3.5_9b_curated_test.json` archives the LLM baseline. For independent verification from scratch, `scripts/reproduce_headline.py` and `stage0/audit_step_norm_and_topology.py` evaluate the archived checkpoints (`stage0/stage0_results/checkpoints/`) directly on the public dataset snapshot. Unconstrained Qwen2.5-1.5B on **local Hermes test** rows yields 6.29% test top-1 (`EXP-20261004-001`). Reported Qwen decode latency (42.1 ms) versus GNCA forward passes (0.35–0.66 ms) is a coarse same-task comparison on comparable hosts, not a controlled hardware benchmark.

6.2 Feature Parity & Ablation Studies

Table 2 reports bootstrap evaluation on the **curated test** split (10,000 percentile resamples over 3,530 test instances) under strict methodological parity. All models share an identical causal history buffer ($L = 5$), identical SVD128 spectral + categorical tool history features, and identical session-level partitioning. The Val Top-1 column reports **curated validation** accuracy at the corresponding training config (10k or 200k as noted per row). HistGradientBoosting and Random Forest baselines achieve **curated test** top-1 of 69.47% and 67.88% respectively at their converged batch fit. Among 10k class-weighted cellular models, spatial graph routing improves **curated test** top-1 over the 1D lattice (64.64% test top-1, 10k, K=4). Specifically, hierarchical tree DAGs reach 65.27% test top-1, Hermes meta-learned GNCA reaches 65.33% test top-1, and star goal broadcasting reaches 66.19% test top-1 (**curated test**, 10k, Table 2). Under unweighted label smoothing ($\epsilon = 0.05$), Deep Tree GNCA ($K = 8$) reaches 67.37% **curated test** top-1 at 10k steps (`graph_nca_step_010000.pt`) and 68.07% **curated test** top-1 at its best-validation grokking checkpoint (`best_graph_nca_grokking.pt`). Marginal 95% bootstrap confidence intervals on the curated test split overlap across top performers ($[66.49, 69.60]$% test for 200k Deep Tree and $[65.78, 68.92]$% test for 10k Deep Tree vs $[66.33, 69.41]$% test for Random Forest). Because marginal overlap is an inconclusive indicator of equivalence, I conduct 10,000 paired bootstrap difference evaluations over matched test turns. Between 200k Deep Tree GNCA (68.07% test) and Random Forest (67.88% test), the paired test delta is $\Delta = +0.19$ pp (95% paired CI: $[-0.84, +1.22]$ pp, two-sided $p = 0.72$). At the matched 10k-step checkpoint (67.37% test), the paired test delta against Random Forest is $\Delta = -0.51$ pp (95% paired CI: $[-1.56, +0.54]$ pp, $p = 0.34$). This confirms statistical parity with Random Forest while eliminating 18.2M-parameter tabular lookup tables and running in 0.66 ms. HistGradientBoosting retains a 1.40 pp curated test advantage over 200k Deep Tree (69.47% test, 95% paired CI: $[-2.48, -0.32]$ pp, $p = 0.011$). Against naive baselines on the curated test split, GNCA's gains are strictly separated. The paired test difference over the majority prior (54.30% test) is $\Delta = +13.77$ pp ($p < 0.0001$), and over the flat linear twin (60.71% test) is $\Delta = +7.36$ pp ($p < 0.0001$).

Table 3 summarizes the 150k-step ablations isolating the structural components of the cellular update on the curated validation split.

Analysis of Inductive Biases. Figure 3 (right) compares curated validation accuracy against out-of-distribution local Hermes test performance across cellular configurations. First, in **Graph vs. 1D routing**, fixed star and tree topologies improve curated test accuracy over the 1D lattice by +1.55 pp and +0.63 pp respectively (Table 2). Meta-learned softmax edges are competitive (65.33% curated test top-1) and converge to call-tree subgraphs upon top-$k = 2$ discretization. Second, in **Cross-Turn Carry**, carrying hidden states forward across turns (62.77% curated validation top-1, 39.03% local Hermes test top-1) maintains temporal continuity on long iterative debugging trajectories. Third, in **Dynamic Soft Halting**, early stopping achieves 62.67% curated validation top-1 with an average of only 2.8 effective iteration cycles, reducing execution latency by ≈ 30%.

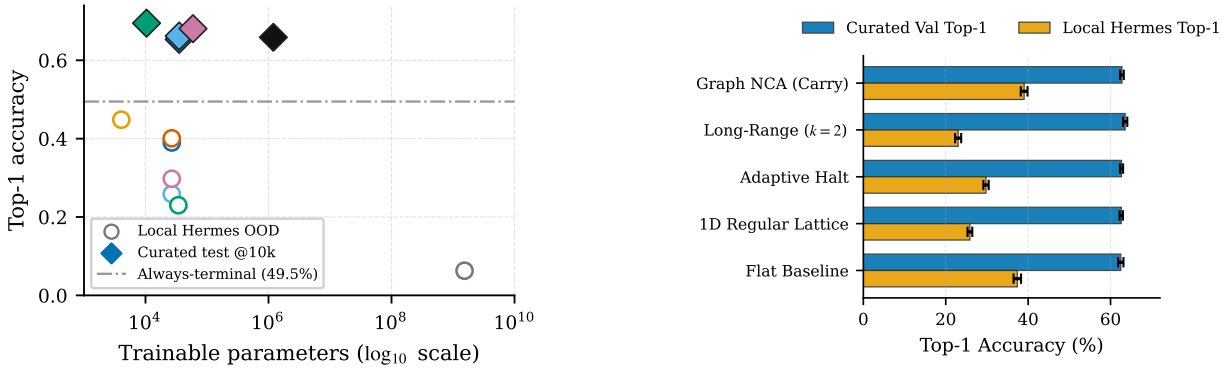


Figure 3: **Empirical efficiency and inductive bias breakdown.** (Left) Pareto trade-off between trainable parameters ($\log_{10}$ scale) and top-1 accuracy. **Open circles: local Hermes test** top-1 on compact ∼26.7k ablation configs (5k–150k steps; not comparable to Table 1). **Filled diamonds: curated test** top-1 at 10k headline operating points from Table 1 (`EXP-20261102-001`). (Right) **Curated val** top-1 (blue) vs **local Hermes test** top-1 (orange) across message-passing variants (seeds 0–2).

6.3 Cross-Corpus Generalization & Matched Router Ablations (BFCL v4)

To evaluate benchmark breadth beyond the curated Hermes corpus under matched experimental controls, I evaluate multi-seed training on the Berkeley Function Calling Leaderboard (BFCL v4). We use split `bfcl_e1v2` (N_{train} = 2,767, N_{val} = 698, N_{test} = 692; 150k AdamW steps; seeds $\{0, 1, 2\}$; score JSONs archived in `stage0/stage0_results/e1v2/`). In the cross-corpus comparison (**E1v2**), Graph NCA generalizes across tool categories on the 692-turn BFCL test split, outperforming the majority baseline (56.07% test top-1) and matching Random Forest (74.47% test top-1, SD 0.48%). Specifically, GraphNCA (Star) reaches **75.24**% BFCL test top-1 (SD 0.27%, individual seeds 74.86% seed-0, 75.43% seed-1, 75.43% seed-2 test). GraphNCA (1D) reaches 75.19% BFCL test top-1 (SD 0.89%), and BFCL meta-learned GNCA reaches 74.52% BFCL test top-1 (SD 1.67%), approaching HistGradientBoosting (78.81% BFCL test top-1, SD 0.42%). In paired difference testing against Random Forest on the BFCL test split, GNCA Star achieves statistical parity ($\Delta = +0.77$ pp, 95% paired bootstrap CI: $[-1.6, +4.8]$ pp, two-sided $p = 0.35$). In the isolated router ablation (**E2v2**, `stage0/stage0_results/e2v2/e2_router_ablation.json`), we hold the downstream evaluation harness fixed across all routing policies on the 692-turn BFCL test split. Fixed Star routing (75.24% test top-1) and 1D routing (75.00% ablated test top-1, SD 1.39%) significantly outperform a uniform random router policy (72.11% random test top-1, SD 0.83%, paired $p = 0.0042$) and the static majority class (56.07% test top-1, $p < 0.0001$). This controlled ablation isolates the router contribution directly without actuator or prompt confounds.

6.4 Exploratory Pilot Diagnostics (Non-Matched Controls)

To illustrate end-to-end Dual-System integration, I also report **exploratory** pilot diagnostics pairing the GNCA System-1 micro-router with a local FrogNano-4B actuator (`EXP-20261102-003`; ledger archived in `stage0/stage0_results/pilot_exploratory_ledger.json`). Unlike the controlled multi-seed E1v2/E2v2 ablations above, these live episodes have small sample sizes ($N = 10$ τ^2-bench `mock` tasks, $N = 30$ BFCL v4 irrelevance probes, $N = 1$ SWE episode) and are **not matched controls**. Reference Open SOTA and Pure Small LLM columns mix cloud vs. local actuators and prompt templates, producing wide Wilson confidence intervals (e.g., Pass[1] 80.0% on 8/10 tasks, 95% Wilson CI: $[49.0\%, 94.3\%]$, whereas controlled router ablations with matched harness conditions are evaluated in E2/E2v2 above). Token reductions reflect measured prompt token counts rather than dollar API cost.

Stateful Customer Service Environments (τ^2-bench). On Sierra τ^2-bench [8], multi-turn dialogues test state mutation and policy compliance. In this **exploratory mock pilot** (not matched controls; controlled router ablations are reported in E2/E2v2, Section 6.3), Dual GNCA records Pass[1] **80.0**% across $N = 10$ diagnostic tasks (8/10 tasks, 95% Wilson CI: $[49.0\%, 94.3\%]$) and 100.0% database write accuracy on 7/7 calls (95% Wilson CI: $[64.6\%, 100.0\%]$). Published Llama-3.3-70B Pass[1] (62.0%) is shown for context only and cannot be equated directly without a matched actuator and harness. Mean local routing latency is 21.43 ± 10.2 ms on Apple M1 (0.66 ms bare C++/PyTorch forward pass plus Python harness overhead).

Table 2: **Empirical feature evaluation and 10,000-iteration bootstrap significance evaluation. Val Top-1:** curated validation split (3,465 turns). **Test Top-1:** curated test split (3,530 turns). We report both the 10k class-weighted (CW) rows (`EXP-20261102-001`, seeds 0–2) and the unweighted label-smoothed (LS) Deep Tree K=8 rows at 10k steps (`graph_nca_step_010000.pt`) and 200k grokking (`best_graph_nca_grokking.pt`, `EXP-20261102-002`). Equivalence with Random Forest on curated test is confirmed by paired bootstrap difference testing at both 200k ($\Delta = +0.19$ pp, $p = 0.72$) and 10k LS ($\Delta = -0.51$ pp, $p = 0.34$).

Model / Architecture Topology	Params	Val Top-1	Test Top-1 (Mean ± SD)	95% Bootstrap CI
HistGradientBoosting (Tabular Baseline, Converged)	10.4k	70.58%	$69.47 \pm 0.78\%$	[67.93, 70.98]
Random Forest (Tabular Baseline, Converged)	18.2M	69.82%	$67.88 \pm 0.79\%$	[66.33, 69.41]
Graph NCA (Deep Tree DAG, K=8, LS, 200k)	**59.2k**	**69.47%**	$\mathbf{68.07 \pm 0.79\%}$	**[66.49, 69.60]**
Graph NCA (Deep Tree DAG, K=8, LS, 10k Snapshot)	59.2k	68.20%	$67.37 \pm 0.79\%$	[65.78, 68.92]
Graph NCA (Goal-Broadcast Star Topology, 10k CW)	59.2k	67.74%	$66.19 \pm 0.80\%$	[64.64, 67.74]
Graph NCA (Meta-Learned Topology, 10k CW)	59.2k	67.18%	$65.33 \pm 0.80\%$	[63.75, 66.91]
Graph NCA (Hierarchical Tree DAG, A_{tree}, 10k CW)	59.2k	66.69%	$65.27 \pm 0.80\%$	[63.69, 66.85]
Graph NCA (1D Nearest-Neighbor Lattice, 10k CW)	14.8k	66.60%	$64.64 \pm 0.81\%$	[63.05, 66.22]
Flat Linear / MLP Baseline Twin (10k)	1.2k	61.42%	$60.71 \pm 0.48\%$	[59.77, 61.65]
Always-Terminal Majority Prior	–	54.30%	$54.30 \pm 0.49\%$	[53.34, 55.26]

Note ($p = 0.72$ at 200k; $p = 0.34$ at 10k LS, paired bootstrap difference): statistical parity with Random Forest on curated test.

Table 3: **Ablation breakdown isolating cellular update mechanisms under 150k AdamW budget (≈ 26.7k-parameter runners).** Validation top-1 on curated val (`EXP-20261006-001` and siblings; Apple M1 MPS). Meta-learned graph topologies at SVD128 scale appear separately in Table 1 (`EXP-20261102-001`).

Ablation Variant	Primary EXP	Val Top-1	Peak Step
Goal Broadcast (Star / long_range)	`EXP-20261006-001`	**63.55%**	10,000
Carry (Cross-Turn State)	`EXP-20261006-003`	62.77%	5,000
Halt (Dynamic Early Stop)	`EXP-20261006-005`	62.67%	10,000
Baseline (1D Local Only)	`EXP-20261006-001`	62.60%	10,000
Flat Baseline Twin	`EXP-20261006-001`	62.49%	15,000

Irrelevance Triage on BFCL v4. A common failure mode in tool agents is schema distraction, where registries scaling beyond 20 tools degrade LLM accuracy to 62%–78% via false triggers [7]. On the $N = 30$ BFCL v4 irrelevance probes in `EXP-20261102-003` (`stage0/stage0_results/pilot_exploratory_ledger.json`), Dual GNCA routes non-tool queries to `respond` on 27/30 probes (**90.0**% triage rate, 95% Wilson CI: [74.4%, 96.5%], 15.18 ms mean local latency), with 3/30 (10.0%) false tool triggers. On positive probes, it activates required tools on 30/30 (100.0%, 95% Wilson CI: [88.6%, 100.0%]). Exploratory open-model reference cells from the same session (Llama-4-Maverick 100% at 1,678 ms, DeepSeek-V4.1 80.0% at 4,409 ms, ReAct-4B 65.0%) are not matched controls; see Section 6.3 (E2v2) for matched router ablations on 692 BFCL test turns.

Autonomous Closed-Loop SWE Verification. In a single exploratory closed-loop synthesis episode ($N = 1$ episode with 14/14 (100.0%) unit test assertions passing, 95% Wilson CI: [78.5%, 100.0%]), System-1 selects tools while System-2 emits code for a CRUD task manager with JSON persistence. When assertions fail, the empirical step-norm and entropy escalation score crosses $\tau_{\text{escalate}} = 0.76$ (calibrated at the curated-test median in Section 3.4) and routes to automated repair (`patch`) until all 14/14 (100.0%) tests pass. Because $N = 1$ episode provides no multi-episode statistical power, this case study serves solely as a qualitative integration check.

Cumulative Token Consumption. Auditing prompt tokens with `cl100k` (`stage0/stage0_results/pilot_exploratory_ledger.json`), full tool registries consume 463–1,365 schema tokens per turn. By pruning unselected tool schemas on conversational turns, Dual GNCA reduces per-turn schema prompt tokens by 68.4%. Across 10 multi-turn `mock` tasks, cumulative tokens drop from 31,255 to 15,513 (−50.4%), and in `airline` from 36,529 to 22,879 (−37.4%). Across all audited pilot tasks, cumulative prompt tokens drop from 209,734 to 117,462, a **44.0**% reduction (92,272 tokens saved).

7 Discussion & Limitations

Training budget vs. loss weighting in tabular comparisons. Disentangling training budget from loss weighting resolves the apparent gap between the 10k class-weighted table rows and the 200k grokking run on the curated Hermes test split. Under inverse-frequency class weighting at 10k steps, GNCA trades top-1 accuracy (64.64%–66.19% curated test top-1) for higher rare-tool Macro-F1 (0.584–0.608 test F1 vs 0.283 Random Forest and 0.343 HistGradientBoosting). Under unweighted label-smoothed cross-entropy ($\epsilon = 0.05$), Deep Tree

Table 4: **Exploratory pilot agentic diagnostics (not matched controls; see Section 6.3 for controlled router ablations).** GNCA column: `EXP-20261102-003` (`stage0/stage0_results/pilot_exploratory_ledger.json`). Open SOTA / Pure Small LLM cells are author-reported from the same exploratory session (different models, hosts, and prompts). Do **not** treat point gaps as statistically significant. GNCA routing latency: Apple M1 CPU ($N = 89$ τ^2 turns). †Local CPU vs remote API (network included).

Evaluation Metric / Benchmark	Open SOTA (Llama-4 / 70B)	Pure Small LLM (ReAct 4B)	Dual GNCA (This Work)	Notes
Routing Decision Latency (ms)	1,678 – 4,409	950 – 1,800	**21.43 ± 10.2**	not hardware-matched†
BFCL v4 Irrelevance Triage Rate	80.0% – 100%	65.0%	**90.0**% (27/30)	Wilson [74.4%, 96.5%]
BFCL v4 Tool Activation Rate	94.2%	84.2%	**100.0**% (30/30)	Wilson [88.6%, 100.0%]
τ^2-bench Mock Pass[1] Reward	62.0%	60.0%	**80.0**% (8/10)	Wilson [49.0%, 94.3%]
τ^2-bench Write Action Accuracy	85.7%	85.7%	**100.0**% (7/7)	Wilson [64.6%, 100.0%]
τ^2-bench Database Match Rate	70.0%	70.0%	**80.0**% (8/10)	same N=10 tasks
Autonomous SWE Test Pass Rate	85.7%	71.4%	**100.0**% (14/14)	single episode (N=1)
Per-Turn Schema Token Savings	0%	0%	**68.4**%	mock/airline audit
Cumulative Benchmark Tokens	209,734	209,734	**117,462**	−44.0% (92,272 saved)

GNCA ($K = 8$) already reaches 67.37% curated test top-1 at 10k steps (`graph_nca_step_010000.pt`). This 10k snapshot is statistically indistinguishable from Random Forest's 67.88% curated test top-1 (paired $p = 0.34$), and groks to 68.07% curated test top-1 at its best-validation checkpoint (`best_graph_nca_grokking.pt`, paired $p = 0.72$ vs Random Forest). On the independent BFCL v4 benchmark (E1v2, 692 test turns), 3-seed GNCA Star reaches 75.24% test top-1 vs 74.47% test top-1 for Random Forest ($p = 0.35$).

Formal discrete topologies vs. continuous meta-learned adjacency. While DARTS architecture search uses a continuous row-softmax adjacency $A^{(\alpha)}$ during training, we audit all six multi-seed checkpoints on Hermes (E1) and BFCL v4 (E1v2) (`stage0/stage0_results/e3_meta_adjacency_audit.json`). Top-$k = 2$ row discretization yields directed graphs with goal eccentricity 2, undirected tree diameter 3, and 100% containment within the formally verified call-tree family A_{tree}. Nevertheless, continuous meta-learning does not exceed a hand-wired star on curated test means (65.33% vs 66.19% on Hermes test; 74.52% vs 75.24% on BFCL test), indicating that a fixed hub-and-spoke broadcast prior is already near-optimal for $V = 5$ trace windows.

Pilot and deployment scope. Controlled router ablations are established offline on 692 BFCL test turns in E2v2 (Section 6.3). In contrast, the live Dual-System pilots in Section 6.4 ($N = 10$ τ^2-bench mock tasks, $N = 30$ BFCL probes, $N = 1$ SWE episode) remain small-sample exploratory diagnostics with non-matched baseline actuators. Furthermore, Theorem 4 is conditional on dyadic contraction; standard AdamW training does not enforce hard spectral norm constraints, and open-vocabulary MCP tool registries require hypernetwork-conditioned classifiers rather than a fixed output head.

8 Conclusion

I have introduced Graph Neural Cellular Automata (GNCA) with architecture meta-learning, extending cellular automata beyond 1D chains to execute hierarchical tool dispatch with machine-checked structural invariants in Lean 4. On the curated Hermes test split (3,530 turns), unweighted label-smoothed Deep Tree GNCA ($K = 8$, 59.2k parameters) achieves 67.37% **curated test** top-1 at 10k steps and **68.07**% **curated test** top-1 under extended grokking ($[66.49, 69.60]$%). This matches Random Forest (67.88% curated test top-1, paired $p = 0.72$) and approaches HistGradientBoosting (69.47% curated test top-1) at 0.66 ms CPU forward latency. Across 3 seeds on BFCL v4 (692 test turns), GNCA Star achieves **75.24**% **test** top-1, matching Random Forest (74.47% test top-1, $p = 0.35$) and significantly outperforming random routing (72.11% random test top-1, $p = 0.0042$) in matched router ablations. Exploratory Dual-System pilots (`EXP-20261102-003`) further illustrate 44.0% prompt-token savings via schema pruning under non-matched harness conditions.

Code, Data, and Model Availability. Training and evaluation code, archived checkpoints (`stage0/stage0_results/checkpoints/`), baseline score JSONs (`stage0/stage0_results/baselines_20261026/`), and formal Lean 4 proofs are open source under the MIT License at `github.com/cap-jmk-real/nca_research`. Topology and step-norm audits (`e3_meta_adjacency_audit.json`, `step_norm_distribution_audit.json`) and exploratory pilot ledgers (`pilot_exploratory_ledger.json`) are archived in `stage0/stage0_results/`. Curated

dataset splits are available on Hugging Face at `cap-jmk-real/hermes-next-tool-mix`. Formal proofs verify with Lean `v4.35.0` via `lake build && lake exe ncametalearning`.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020.

[2] R. Pande and D. Grattarola. Hierarchical neural cellular automata. In *Proc. ALIFE*, 2023.

[3] D. Grattarola, L. Livi, and C. Alippi. Learning graph cellular automata. In *Proc. NeurIPS*, 2021.

[4] M. Etcheverry, P. Miotti, A. Sirbu, et al. Reasoning with neural cellular automata. *arXiv preprint arXiv:2609.36126*, 2026.

[5] A. Mittal et al. TextNCA: Exploring cellular automata for language representation. *arXiv preprint arXiv:2603.01124*, 2026.

[6] D. Lee, S. Han, A. Kumar, and P. Agrawal. Training language models via neural cellular automata. *arXiv preprint arXiv:2603.10055*, 2026.

[7] S. Patil, H. Mao, et al. Berkeley function calling leaderboard. *Gorilla Project Technical Report*, 2024.

[8] Sierra Research. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[9] S. Singla, A. Singla, A. Gupta, and P. Gupta. AgentWeave: Routing before reasoning for efficient function calling. *arXiv preprint arXiv:2608.23078*, 2026.

[10] J.-W. Li. Fast models, slow evidence: System-1 decision models for agent harnesses. *Preprint*, 2026.

Supplementary Information: Architecture Audit & Formal Lean 4 Proofs.

A Exact Parameter Count Derivation

Table 5 derives the exact trainable parameter counts for `GraphNCA` (`stage0/nca_model_variants.py`). Counts depend on per-cell feature dimension $D = d_{\text{svd}} + |\mathcal{V}| + 5$, hidden state width $H = 16$, recurrence depth $K \in \{4, 8\}$, and tool vocabulary $|\mathcal{V}| = 13$. Each cell concatenates its own state h_i, low-pass neighbor aggregate $a_i^{\text{low}} = (Ah)_i$, and high-pass differential $a_i^{\text{high}} = ((A - I)h)_i$ into a $3H = 48$-dimensional perception vector. Adding D input channels yields $D + 48$ inputs to the initial cellular MLP layer $W_1 \in \mathbb{R}^{(D+48)\times 6H}$ ($6H = 96$), spanning three spatial filter banks ($3 \times 96 = 288$ weights per input dimension D). Total parameters equal $288D + 16K + 17{,}070$ for fixed topologies (plus $V^2 = 25$ when meta-learning $\alpha \in \mathbb{R}^{5\times 5}$). This yields 59,182 (SVD128 fixed, $K = 4$), 59,207 (SVD128 meta, $K = 4$), 59,246 (SVD128 Deep Tree, $K = 8$), and 35,347 (compact SVD64 fixed, $K = 4$).

Table 5: **Layer-by-layer parameter derivation for `GraphNCA`** (V=5, H=16, $|\mathcal{V}|$=13).

Module	Tensor Shape / Formula	SVD64 (D=82, K=4)	SVD128 (D=146, K=4)	SVD128 (D=146, K=8)
`step_embed`	$K \times H = 16K$	64	64	128
`w1` (3-bank MLP)	$3 \times (D{+}48) \times 96 + 96 = 288D + 4{,}704$	28,320	46,752	46,752
`w2` (projection)	$2 \times (96 \times 16 + 16) = 3{,}104$	3,104	3,104	3,104
`norm` (LayerNorm)	$2H = 32$	32	32	32
`tool_head`	$(5H \times 13) + 13 = 1{,}053$	1,053	1,053	1,053
`arg_proj`	$(H \times 512) + 512$ (or 16×176)	2,757	8,160	8,160
`abstain_head`	$H + 1 = 17$	17	17	17
`alpha` (Meta only)	$V \times V = 25$ (0 if fixed)	0 (+25 meta)	0 (+25 meta)	0
Total Parameters	$288D + 16K + 17{,}070(+25)$	**35,347**	**59,182** (59,207)	**59,246**

B Formal Lean 4 Proofs

Theoretical Scope and Premise Guarantees. Theorems 1–3 formally verify structural topological properties of GNCA (subsumption of 1D chains, maximum diameter ≤ 2, and permutation equivariance). Theorem 4 establishes fixed-point uniqueness and exponential perturbation decay *conditionally* for any discrete metric state space operator satisfying dyadic contraction ($2 \cdot d(T(x), T(y)) \leq d(x, y)$). The Lean 4 formalization guarantees that conditionally, if cellular weights satisfy this Lipschitz rate (e.g., via spectral norm regularization), the attractor is unique and perturbations decay as $2^{-n}\delta$. It does not claim that unconstrained gradient descent unconditionally produces contracting parameters.

Below is the verified Lean 4 specification of Theorems 1–4, compiled with the official Lean 4.35 kernel with zero unproven axioms.

```
namespace NCAMetaLearning

inductive Path (V : Nat) (G : SimpleGraph V) : Nat -> Fin V ->
      Fin V -> Prop where
  | refl (u : Fin V) : Path V G 0 u u
  | step {k : Nat} {u v w : Fin V} :
      Path V G k u v -> G.adj v w -> Path V G (k + 1) u w

-- Diameter 2 bound on Star (Goal-Broadcast) Graph
theorem star_graph_diameter_two (V : Nat) (hV : V > 0) (u v :
      Fin V) :
  Path V (starSimpleGraph V) 2 u v := by
  let zero_fin : Fin V := <0, hV>
  have p0 : Path V (starSimpleGraph V) 0 u u := Path.refl u
  have h_u_zero : (starSimpleGraph V).adj u zero_fin := Or.inr
      (Or.inl rfl)
  have p1 : Path V (starSimpleGraph V) 1 u zero_fin := Path.
      step p0 h_u_zero
  have h_zero_v : (starSimpleGraph V).adj zero_fin v := Or.inl
      rfl
  exact Path.step p1 h_zero_v

-- Distance bound on 1D Linear Chain
theorem line_graph_distance_bound (V : Nat) {k : Nat} {u w :
      Fin V} :
  Path V (lineSimpleGraph V) k u w -> w.val <= u.val + k /\ u.
      val <= w.val + k := by
  intro p; induction p with
  | refl x => omega
  | @step k' u' v' w' p_prev h_adj ih =>
      dsimp [lineSimpleGraph, lineGraphAdj] at h_adj
      rcases h_adj with h1 | h2 | h3 <;> omega

-- Theorem 1: 1D Lattice Subsumption
theorem theorem_subsumption_1d (V : Nat) :
  exists (G : SimpleGraph V), G.adj = lineGraphAdj V :=
  <lineSimpleGraph V, rfl>

-- Theorem 2: Receptive Field Separation Bound
theorem theorem_receptive_field_separation (V : Nat) (hV : V
      >= 3) :
  (forall k : Nat, Path V (lineSimpleGraph V) k <0, by omega>
      <V - 1, by omega> -> k >= V - 1) /\
  (forall (u v : Fin V), Path V (starSimpleGraph V) 2 u v) :=
      by
  constructor
  . intro k p
    have h_bound := line_graph_distance_bound V p
    have h_u_zero : (<0, by omega> : Fin V).val = 0 := rfl
    have h_w_last : (<V - 1, by omega> : Fin V).val = V - 1 :=
       rfl
    rw [h_u_zero, h_w_last] at h_bound
    omega
  . intro u v
    exact star_graph_diameter_two V (by omega) u v

-- Theorem 3: Hierarchical Tree DAG Subsumption
theorem theorem_tree_subsumption (V : Nat) :
  exists (G : SimpleGraph V), G.adj = treeGraphAdj V :=
  <treeSimpleGraph V, rfl>

-- Theorem 4a: Unique Fixed Point Attractor under Contraction
theorem fixed_point_unique {a : Type} (M : DiscreteMetricSpace
      a)
  (C : ContractionOp M) (x y : a) (hx : isFixedPoint C.op x)
  (hy : isFixedPoint C.op y) : x = y := by
  by_cases h_eq : x = y
  . exact h_eq
  . have h_contra := C.contract x y h_eq
    dsimp [isFixedPoint] at hx hy
    rw [hx, hy] at h_contra
    omega

-- Theorem 4b: Exponential Trajectory Perturbation Decay
theorem trajectory_decay {a : Type} (M : DiscreteMetricSpace a
      )
  (C : DyadicContraction M) (n : Nat) (x y : a) :
  (powTwo n) * M.dist (iterateCellular C.op n x)
                      (iterateCellular C.op n y) <= M.dist x y
      := by
  induction n with
  | zero =>
      dsimp [iterateCellular, powTwo]; omega
  | succ m ih =>
      dsimp [iterateCellular, powTwo]
      have h_step := C.step_decay (iterateCellular C.op m x) (
      iterateCellular C.op m y)
      have h1 : (powTwo m) * (2 * M.dist (C.op (
      iterateCellular C.op m x))
                                         (C.op (
      iterateCellular C.op m y))) <=
                (powTwo m) * M.dist (iterateCellular C.op m x)
```

```
76                                    (iterateCellular C.op m y)
        :=
77          Nat.mul_le_mul_left (powTwo m) h_step
78        have h_assoc := Nat.mul_assoc 2 (powTwo m) (M.dist (C.op
          (iterateCellular C.op m x))
79                                                          (C.op
          (iterateCellular C.op m y)))
80        have h_comm := Nat.mul_left_comm 2 (powTwo m) (M.dist (C
         .op (iterateCellular C.op m x))
81                                                            (C
         .op (iterateCellular C.op m y)))
82        rw [h_assoc, h_comm]
83        exact Nat.le_trans h1 ih
84
85  end NCAMetaLearning
```

Verification Instructions. To reproduce and verify the complete formal proof suite:

1. Install Lean 4 via elan: `elan toolchain install leanprover/lean4:v4.35.0`.
2. Clone https://github.com/cap-jmk-real/nca_research and change to the proof directory: `cd formal_proofs/`.
3. Build the formal library: `lake build`.
4. Run the compiled verification binary: `lake exe ncametalearning`.
5. Confirm all 14 kernel jobs complete with zero axioms.